

Contents

1	DD01: アカウント・ユーザー管理	1
1.1	0. 文書情報	1
1.2	1. 対象範囲	1
1.3	2. 収録ロジック・対応章	1
1.4	3. 詳細設計本文	2
1.4.1	3.1 利用者・権限詳細設計（参照のみ）	2
1.4.2	3.2 状態詳細設計（参照のみ）	2
1.4.3	3.3 画面詳細設計（参照のみ）	2
1.4.4	3.4 機能詳細設計（参照のみ）	2
1.4.5	3.5 実装モジュール構成	2
1.4.6	3.6 関連する横断設計	3
1.4.7	3.7 オーナー/メンバー/プロジェクト割当	3
1.4.8	3.8 退会フロー（参照のみ）	3
1.5	4. 関連設計	3
1.6	5. テスト観点	3
1.6.1	5.1 結合テスト例（ロックアウト）	4
1.6.2	5.2 その他観点	4
1.7	6. 未確定事項・確認事項	4

1 DD01: アカウント・ユーザー管理

1.1 0. 文書情報

項目	内容
文書名	DD01: アカウント・ユーザー管理
詳細設計 ID	DD01
対象システム	FAQ AI ウィジェット SaaS / メインシステム
関連機能 ID	FR-001～022（新規登録 / メール確認 / パスワード再設定 / ログイン / ロックアウト / 管理者ユーザー管理 / 再認証 / 利用規約再同意）
作成日	2026-05-17
版数	v1.0
ステータス	承認済

1.2 1. 対象範囲

種別	ID	名称
機能	FR-001～009	新規登録 / メール確認 / パスワード再設定 / ログイン / ログアウト / 再認証 / ロックアウト
機能	FR-015～022	管理者ユーザー登録・停止・削除・有効化 / メール再送 / 退会
機能	FR-011	利用規約再同意フロー
画面	SCR-001	ログイン
画面	SCR-002	新規登録
画面	SCR-003	パスワード再設定
画面	SCR-017	管理者ユーザー一覧
API	/auth/* /admin-users/* /me/email-verification/* /me/terms*	認証認可・管理者ユーザー操作・規約同意
テーブル	accounts sessions access_tokens account_project_grants	アカウント基盤 (account_project_grants.role でプロジェクト別ロール admin / member を管理)

1.3 2. 収録ロジック・対応章

元章	元タイトル	概要
§ 4	利用者・権限詳細設計	権限設計書 / 認証認可設計書への参照（正本は基本設計）

元章	元タイトル	概要
§ 6 SCR-001~003,017	画面詳細設計	ログイン / 新規登録 / パスワード再設定 / 管理者ユーザー一覧 (正本は基本設計)
§ 7	機能詳細設計 (アカウント・ユーザー機能)	認証フロー / 管理者ユーザー管理 / 規約同意 (正本は基本設計)
§ 3.3.1	worker-main-api モジュール構成	routes/auth.ts routes/admin-users.ts routes/email-verification.ts routes/terms.ts

1.4 3. 詳細設計本文

1.4.1 3.1 利用者・権限詳細設計 (参照のみ)

ユーザー種別、3 ロール (オーナー / プロジェクト管理者 / メンバー)、画面別権限マトリクス、認証フロー、Principal 構築は [./02_基本設計/04_権限設計.md](#) および [./02_基本設計/08_認証・認可設計.md](#) を正本とする。

利用者側アカウントは **オーナー** (契約あたり 1 アカウント固定、全権、MVP では譲渡不可、`accounts.owner_account_id = accounts.id` で自己参照、`account_project_grants` には行を持たず全プロジェクトの暗黙の `admin` 相当) と **メンバー** (0..N、`owner_account_id` でオーナーに紐付き、プロジェクトごとに `account_project_grants.role = admin` (プロジェクト管理者) / `member` (メンバー) のいずれかを持つ) に分かれる。オーナー専有機能 (プロジェクト作成・編集・削除、課金、退会、規約再同意、メンバーアカウント完全削除) はプロジェクト別ロールでは付与できない。プロジェクト割当てが 0 のメンバーはダッシュボードのみ利用可能。

1.4.2 3.2 状態詳細設計 (参照のみ)

契約状態 (`accounts.contract_status: active / suspended / deleted_pending / deleted`) の全状態遷移は [./02_基本設計/03_テーブル設計.md](#) § 8 を正本とする。

1.4.3 3.3 画面詳細設計 (参照のみ)

SCR-001 ログイン / SCR-002 新規登録 / SCR-003 パスワード再設定 / SCR-017 管理者ユーザー一覧の表示・入力・操作・バリデーション・モーダル・状態別表示・メッセージ ID は [./02_基本設計/01_画面設計.md](#) および [./02_基本設計/06_メッセージ一覧.md](#) を正本とする。

1.4.4 3.4 機能詳細設計 (参照のみ)

機能別のロジック・データ・API 連動は [./02_基本設計/02_API 設計.md](#) および [./02_基本設計/03_テーブル設計.md](#) を正本とする。本書では実装モジュール構成のみを扱う。

1.4.5 3.5 実装モジュール構成

`app/workers/main-api/src/` 配下に以下のルート / ハンドラ / リポジトリを配置する。

```
src/
├── routes/
│   ├── auth.ts                # POST /auth/login / POST /auth/logout / POST /auth/password-reset
│   ├── admin-users.ts         # GET/POST /admin-users/* (管理者ユーザー CRUD)
│   ├── email-verification.ts  # POST /me/email-verification/{verify,resend}
│   └── terms.ts               # GET /me/terms / POST /me/terms/agree
├── handlers/                  # 上記 routes と 1:1
├── domain/
│   └── contract-status.ts     # contract_status 遷移ガード (active → suspended → deleted_pending → deleted)
├── repository/
│   ├── accounts.ts           # オーナー / メンバー両対応
│   ├── sessions.ts
│   ├── access-tokens.ts
│   └── account-project-grants.ts # account_project_grants(role 列含む) の CRUD
├── middleware/
│   ├── authenticate.ts       # Cookie + CSRF
│   └── require-active-contract.ts
```

```

├── require-terms-agreement.ts
├── require-reauth.ts      # 重要操作再認証
├── csrf.ts
├── lib/
│   ├── argon2id.ts        # hashPassword / verifyPassword
│   └── token.ts           # email_verify / password_reset / activation トークン

```

1.4.6 3.6 関連する横断設計

1.4.6.1 3.6.1 トークン発行・検証 メール確認・パスワードリセット・管理者ユーザー有効化 (activation) の各トークンは [DD08_トークン発行・検証.md](#) の HMAC-SHA256 保存方式を利用する。一回限りトークンは `consumeToken()` を検証直後に必ず呼ぶ。

1.4.6.2 3.6.2 認可ヘルパ `requireTenant / requireProject / requireRole` の各ヘルパは [DD09_認可ヘルパ.md](#) を利用する。

1.4.6.3 3.6.3 監査ログ書込 `auth.signup / auth.login_success / auth.login_failure / auth.logout / auth.password_reset / auth.re_auth.success / auth.ip_blocked / auth.terms_agree` などの action コードはすべて [DD10_監査ログ書込・完全性検証.md](#) の `writeAudit()` で書込み、`retention_class=general` とする。

1.4.6.4 3.6.4 暗号化・鍵管理 `email / phone` などの PII はオーナー派生 で AES-256-GCM 暗号化し、検索用に HMAC-SHA256 列 (`email_hmac` 等) を併設する。詳細は [DD11_暗号化・管理.md](#) を参照。

1.4.7 3.7 オーナー / メンバー / プロジェクト割当

- `accounts.is_owner = 1` がオーナー行、`accounts.owner_account_id = accounts.id` で自己参照
- メンバーは `accounts.owner_account_id` で所属オーナーへ紐付け
- メンバーの権限は `account_project_grants.role(admin/member)` とプロジェクト割当の組合せで管理 (個別権限フラグ方式は MVP で廃止)
- オーナーは `account_project_grants` に行を持たず、全プロジェクトの暗黙の `admin` 相当として扱う
- 共有概念は [../共有/共有概念.md](#) を正本とする

1.4.8 3.8 退会フロー (参照のみ)

- オーナーが退会要求 → `contract_status = 'deleted_pending' + withdrawal_requests` レコード作成 + `scheduled_deletion_at` (30 日後)
- 30 日経過後、§ 14.1.9 定期削除 cron が `contract_status = 'deleted'` に更新
- 詳細は [DD14_バッチ・非同期処理.md](#) を参照

1.5 4. 関連設計

種別	参照先
要件	../01_要件定義/index.md
基本設計	../02_基本設計/index.md
権限設計	../02_基本設計/04_権限設計.md
認証・認可設計	../02_基本設計/08_認証・認可設計.md
共有概念	../共有/共有概念.md
運用設計	../04_運用設計/index.md
将来対応	../05_future/index.md
関連 DD	DD08_トークン発行・検証.md / DD09_認可ヘルパ.md / DD10_監査ログ書込・完全性検証.md / DD11_暗号化・管理.md

1.6 5. テスト観点

AC ID	テスト ID	テスト方式	テストファイル
AC-001	e2e-signup-001	E2E	apps/admin/e2e/auth/signup.spec.ts
AC-002	e2e-email-verify-001	E2E	apps/admin/e2e/auth/email-verify.spec.ts
AC-003	e2e-pwreset-001	E2E	apps/admin/e2e/auth/password-reset.spec.ts
AC-004	e2e-login-001 / e2e-logout-001	E2E	apps/admin/e2e/auth/login.spec.ts / .../logout.spec.ts
AC-005	it-reauth-001	Integration	workers/main-api/test/integration/auth/reauth.test.ts
AC-007	it-lockout-001	Integration	workers/main-api/test/integration/auth/lockout.test.ts
AC-008	it-session-expire-001 / it-session-expire-002	Integration	workers/main-api/test/integration/auth/session-expire.test.ts
AC-009	e2e-admin-user-activation-001	E2E	apps/main/e2e/admin-users.spec.ts
AC-016	e2e-withdrawal-001	E2E	apps/admin/e2e/account/withdrawal.spec.ts
AC-027	e2e-terms-modal-001	E2E	apps/admin/e2e/terms/reaccept.spec.ts
AC-046	it-force-logout-001	Integration	workers/main-api/test/integration/owner/force-logout.test.ts

1.6.1 5.1 結合テスト例（ロックアウト）

```
// tests/integration/auth/login.test.ts
describe('POST /api/v1/auth/login', () => {
  it('正しい資格情報でログイン成功', async () => {
    await seedAccount({ email: 'admin@example.com', password: 'P@ss12345678', role: 'admin' });
    const res = await (await worker).fetch('http://localhost/api/v1/auth/login', {
      method: 'POST', body: JSON.stringify({ email: '...', password: '...' }),
    });
    expect(res.status).toBe(200);
    expect(res.headers.get('Set-Cookie')).toContain('session=');
  });
  it('5 回失敗でロックアウト', async () => {
    for (let i = 0; i < 5; i++) {
      await (await worker).fetch('...', { method: 'POST', body: JSON.stringify({ ..., password: ... })
    }
    const res = await (await worker).fetch('...', { method: 'POST', body: JSON.stringify({ ..., password: ... })
    expect(res.status).toBe(423); // LOCKED_OUT
  });
});
```

1.6.2 5.2 その他観点

観点	内容
単体テスト	contract-status.ts 遷移ガード（全 4 状態 × 全遷移を網羅）
異常系	ロックアウト / 期限切れトークン / リプレイトークン / 規約未同意時の API 拒否
境界値	パスワード 長 12 文字 / 失敗回数 5 回 / セッション TTL 12h
権限	オーナー専有機能（課金 / 退会 / 規約再同意）にメンバーアクセス時 403
性能	POST /api/v1/auth/login p95 < 800ms（Argon2id verify 支配）

1.7 6. 未確定事項・確認事項

確認事項 ID	確認内容	優先度	ステータス
-	v1.0 リリース時点で全項目確定済み	低	確認済